

Lab 1: Arithmetic Modules: Adder

Using Logisim

Logisim is a Java application, it can be obtained from <http://sourceforge.net/projects/circuit/>.

Simply download and run it (tutorial is available help->tutorial).

Exercise : Inverter (NOT gate)

To build a CMOS inverter, you will need the following Logisim components: • A power source. • A ground. • A P-type transistor, facing south. • An N-type transistor, facing north. • A constant generator, to provide an input value. • A probe to read the output value.

If you follow the steps below and build a circuit, you should end up with a working inverter.

1. Place a P-type transistor, facing south, on the drawing area.
 2. Place an N-type transistor, facing north, below the P-type. You will notice the P-type has a small round circle at its gate and a directional arrow pointing down, whilst the N-type has no circle, and a directional arrow pointing up.
 3. Now place a power component above the top transistor.
 4. Next, place a ground component below the bottom transistor.
 5. Place a constant component to the left of the transistors (or a pin input), this is equivalent to the input A
 6. Place a probe on the right of the circuit. This is equivalent to the output Z
 7. Connect the components with wires
- (see Figure)

Now, by clicking on the constant component and pressing either the 1 or 0 key on your keyboard, (by clicking on the pin if you used a pin input), you can change the logic value that it generates. At the same time, you should see the value at the probe change, in accordance with the logic function NOT. If your circuit does not work as expected, make sure the components are wired correctly, and that the type and orientation of your transistors are correct. If they are not correct, you may see X at the output for a particular input value. Save your work before continuing.

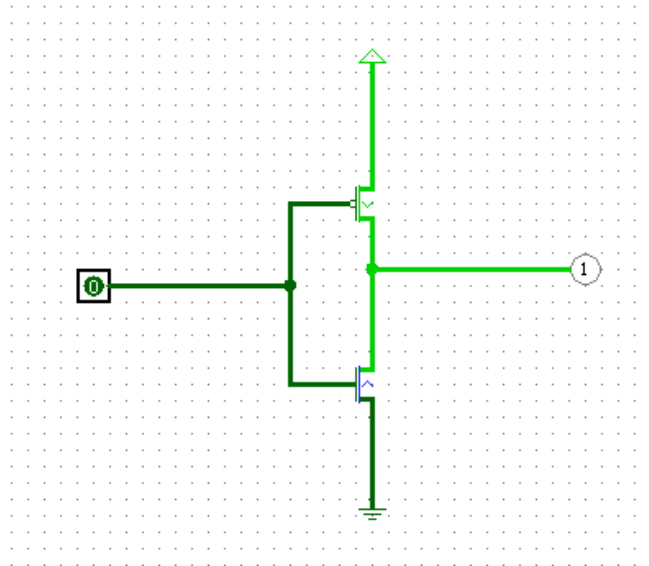


Fig: A CMOS inverter

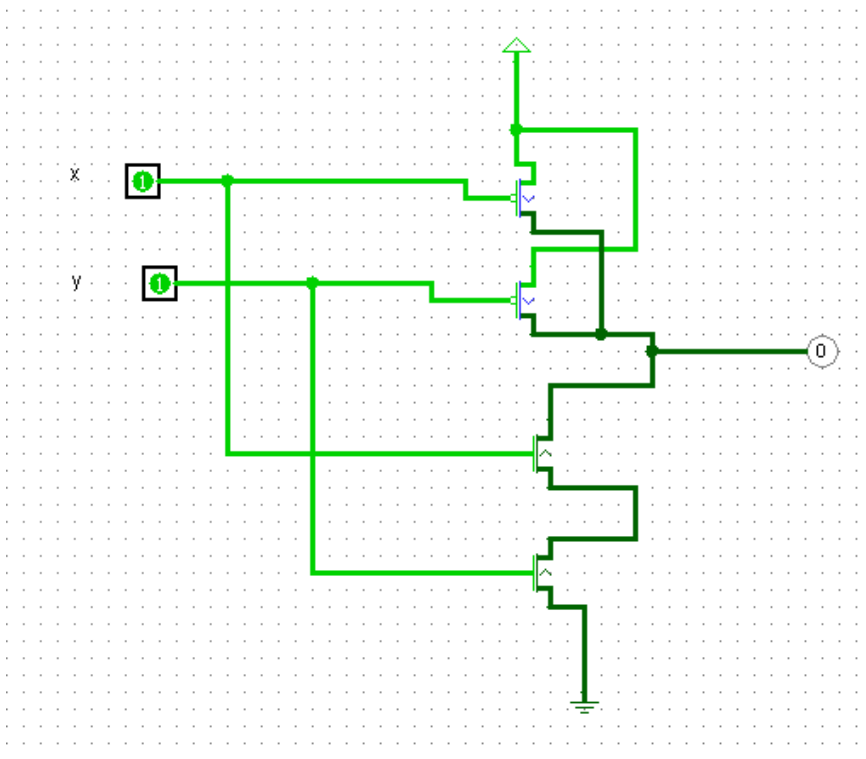


Fig: A CMOS NAND GATE

Assignment:

Using the same methodology demonstrated for the gates shown, develop a gate library that includes **INV, NAND, XOR, Half Adder, and Full Adder**.

- design and implement a **32-bit adder** using this library.
- extend the architecture to support the addition of **5 numbers**, and verify the design using an **appropriate test bench**.